



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware – Unidade 5

Dr. Ney Laert Vilar Calazans

Última alteração: 06/11/2025

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

PROGRAMAÇÃO EM LINGUAGEM DE MONTAGEM DO MIPS (MIPS-S)

INTRODUÇÃO

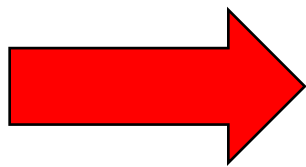
- Conjunto de instruções para estudo
 - ✓ MIPS-2000, usem o subconjunto de instruções definido para a arquitetura MIPS-S
- Arquitetura baseada no paradigma RISC
- Ambiente de Simulação
 - ✓ MIPS Assembler and Runtime Simulator, site
 - <http://courses.missouristate.edu/KenVollmar/MARS>
 - ✓ Comando para executar o ambiente
 - `java -jar Mars4_5.jar`
- Objetivo deste material
 - ✓ Revisar conceitos básicos de programação no processador MIPS (MIPS_S)

COMANDO IF-THEN-ELSE DA LINGUAGEM C

IF THEN ELSE

➤ Mapeando linguagem de alto nível em Linguagem de Montagem (*Assembly*)

```
main() {  
    int i=4, j=6;  
  
    if(i==j)  
        i=i+2;  
    else  
        j=j-1;  
}
```



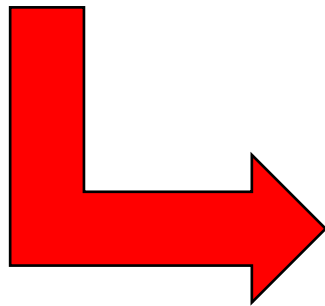
```
.text  
.globl main  
main:  
    li    $t0, 4           # i($t0) ← 4  
    li    $t1, 6           # j($t1) ← 6  
    beq   $t0, $t1, SeEntao # if (i==j)  
    subi  $t1, $t1, 1       # j ← j-1  
    j     fim  
SeEntao:  
    addi  $t0, $t0, 2       # i ← i+2  
fim:  
    jr    $ra
```

COMANDOS DE REPETIÇÃO EM C

COMANDOS DE REPETIÇÃO

Exemplo 1 (laço for)

```
main() {  
    int i;  
    int sum = 0;  
    for(i=0; i<=100; i=i+1)  
        sum += i * i;  
}
```

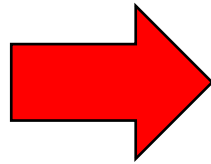


```
.text  
.globl main  
main:  
    move $t0, $zero      # sum ← 0;  
    move $t1, $zero      # i ← 0;  
    li   $t2, 100        # limite superior do for  
  
loop:  
    bgt  $t1, $t2, afterLoop # teste inverso, i>100  
    mul  $t3, $t1, $t1     # i * i  
    add  $t0, $t0, $t3     # sum = sum + i * i;  
    add  $t1, $t1, 1       # i=i+1  
    j    loop             # volta p/ loop  
  
afterLoop:  
    jr   $ra
```

COMANDOS DE REPETIÇÃO

Exemplo 2 (laço do...while)

```
main() {  
    int index = 50;  
    int sumOfValues = 0;  
    do {  
        sumOfValues += index;  
        index = index - 1;  
    } while (index > 0);  
}
```



```
.text  
.globl main  
  
main:  
    li    $t0, 50          # index ($t0) ← 50;  
    move  $t1, $zero       # sumOfValues ($t1) ← 0;  
  
loop:  
    add   $t1, $t1, $t0     # sumOfValues += index  
    addi  $t0, $t0, -1      # index--  
    bgtz  $t0, loop        # i > 0, volta a executar  
  
jr $ra
```

CHAMADAS DO SISTEMA NO SIMULADOR MARS

CHAMADAS DO SISTEMA

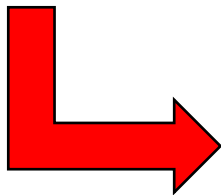
- O código da chamada de sistema deve ser escrito no registrador **\$v0** antes da **syscall**

Serviço	Código em \$v0	Argumento(s)	Resultados
print_int	1	\$a0 – inteiro a imprimir	
print_float	2	\$f12 – <i>float</i> a imprimir	
print_double	3	\$f12-\$f13 – <i>double</i> a imprimir	
print_string	4	\$a0 – endereço da <i>string</i> a imprimir	
read_int	5		\$v0 – inteiro lido (do teclado)
read_float	6		\$f0 – <i>float</i> lido (do teclado)
read_doubl	7		\$f0-\$f1 – <i>double</i> lido (do teclado)
read_string	8	\$a0 – endereço da <i>string</i> lida do teclado, \$a1 – comprimento da <i>string</i> lida	
sbrk/malloc	9	\$a0 – quantidade de memória a alocar (em bytes)	\$v0 – endereço do primeiro byte alocado
exit	10	\$v0 – o código devolvido	

CHAMADAS DO SISTEMA

Exemplo 1 (printf)

```
main() {  
    int x=5, y=3;  
  
    if((x+y)%2==0)  
        x=x+y;  
    else  
        x=y;  
  
    printf("O valor de  
        x é %d", x);  
}
```



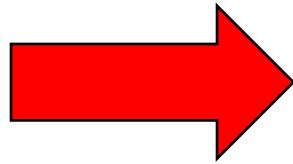
```
.data  
    #declaração das variáveis  
    x: .word 5  
    y: .word 3  
    #declaração do texto  
    texto: .asciiz "O valor de x é "  
.text  
    .globl main  
  
main:  
    lw    $t0, x($zero)    #x=5 - pseudo  
    lw    $t1, y($zero)    #y=3 - pseudo  
    add   $t2, $t0, $t1    #x+y  
    li    $t3, 2  
    ...
```

```
    rem   $t3, $t2, $t3    #(x+y)%2  
    beqz  $t3, IgualZero  #se == 0  
    move  $t0, $t1        #senao  
    j     printCoisa  
  
IgualZero:  
    add   $t0, $t0, $t1  
  
printCoisa:  
    li    $v0, 4  
    la    $a0, texto  
    syscall    #imprime texto  
    li    $v0, 1  
    move  $a0, $t0  
    syscall    #imprime inteiro  
    li    $v0, 10  
    syscall
```

CHAMADAS DO SISTEMA

Exemplo 2 (Resolução interativa)

```
main() {  
    int x;  
    scanf("%d", &x);  
    printf("O valor lido  
    é %d", x);  
}
```



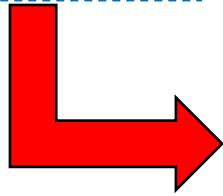
```
.data  
    txtLido: .asciiz "O valor lido é "  
.text  
    .globl main  
main:  
    li $v0, 5  
    syscall          #lê inteiro  
    move $t0, $v0  
    li $v0, 4        #imprime texto  
    la $a0, txtLido  #endereço do texto  
    syscall  
    li $v0, 1  
    move $a0, $t0  
    syscall          #imprime valor lido  
    li $v0, 10  
    syscall
```

VETORES LINEARES

VETORES

Exemplo 1 (Declarando e manipulando vetores)

```
main() {  
    int x[5]={25, 33, 18, 23,  
            32};  
    for(int i=0; i<5; i++)  
        printf("%d", x[i]);  
}
```

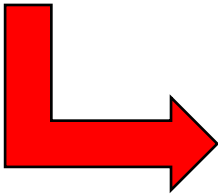


```
.data  
    x: .word 25 33 18 23 32      # Reserva e inicializa 5 palavras em memória  
                                   # Adicionalmente, inicializa os campos da memória  
  
.text  
.globl main  
main:  
    li    $t0, 0                # i ← 0  
    li    $t1, 20               # Define última posição do vetor  
  
loop:  
    beq   $t0, $t1, dim         # Se é o fim do laço, sai  
    li    $v0, 1                # Define o serviço  
    lw    $t2, x($t0)           # Carrega o valor da memória para o registro $t2  
    move  $a0, $t2              # Define a posição do vetor a ser impresso  
    syscall                     # Imprime o valor de i  
    addi  $t0, $t0, 4           # i++, alinhado com a memória 4 bytes por palavra  
    j     loop                  # Volta para testar se saída do loop  
  
fim:  
    li    $v0, 10               # Serviço para finalizar o programa  
    syscall
```

VETORES

Exemplo 2 (Declarando e manipulando vetores)

```
main() {  
    int x[10];  
    for(int i=0; i<10; i++)  
        scanf("%d", &x[i]);  
}
```



```
.data  
    x:      .space 40  # Reserva 40 bytes na memória  
    # isto é quase equivalente a (a primeira opção não escreve nada em x; pode ter lixo anterior)  
    # x:      .word 0:10 # Reserva 10 palavras de memória e inicializa campos com 0  
.text  
.globl main  
main:  
    li      $t0, 0      # i ← 0  
    li      $t1, 40     # Define última posição do vetor  
loop:  
    beq     $t0, $t1, fim      # Se é fim do laço, sai  
    li      $v0, 5        # Lê caractere do teclado  
    syscall  
    sw      $v0, x($t0)  
    addi    $t0, $t0, 4      # i++, alinhado com a memória 4 bytes por palavra  
    j      loop  
fim:  
    li      $v0, 10  
    syscall
```

SUPOORTE A SUBROTINAS

SUORTE A SUBROTINAS

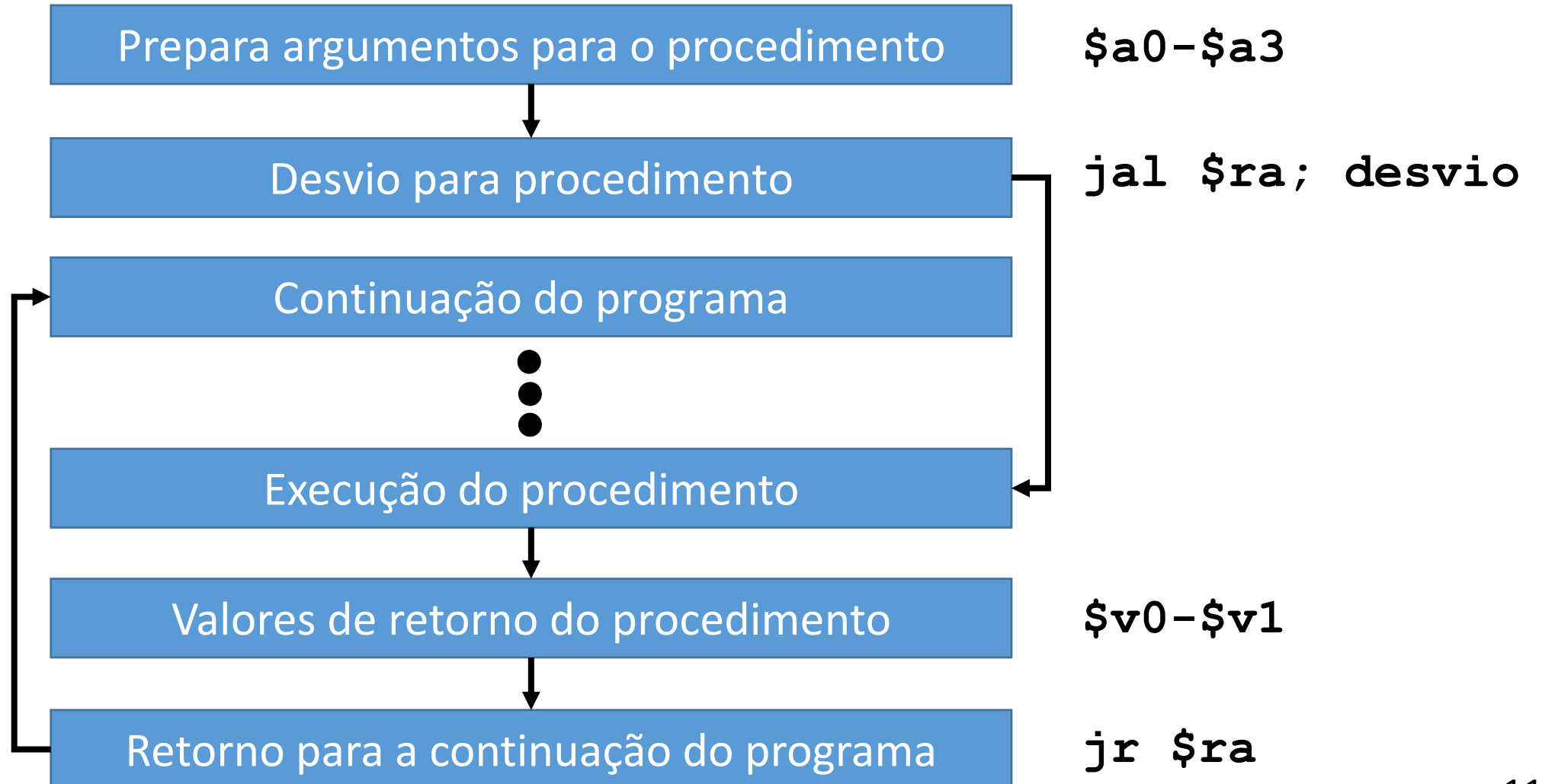
➤ Qual o lugar mais rápido em que se pode armazenar dados a serem usados por uma subrotina?

✓ **Registradores!**

Convenções de software de uso de registradores MIPS para sub-rotinas (compiladores usam estas)

- ✓ \$a0 - \$a3 → parâmetros para a subrotina
- ✓ \$v0 - \$v1 → valores de retorno da subrotina
- ✓ \$ra → registrador de endereço de retorno ao ponto de origem (ra → *return address*)

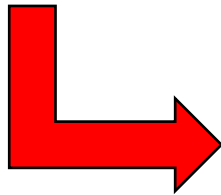
SUPOORTE A SUBROTINAS



CHAMADA DE SUBROTINAS

Exemplo 1 (Passando argumentos)

```
main() {  
    int index;  
    scanf("%d", &index);  
    ImprimeValor(index);  
}  
  
ImprimeValor(int _parametro){  
    printf("Imprimindo valor %d", _parametro);  
}
```

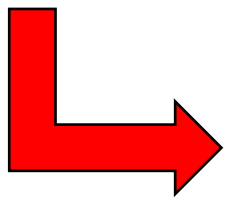


```
.data  
    txtImpressao: .asciiz "Imprimindo valor "  
.text  
    .globl main  
main:  
    move    $s0, $ra        # salva o endereço de retorno  
    li      $v0, 5  
    syscall  
    move    $a0, $v0  
    jal     ImprimeValor    # chama subrotina e salva  
                             #o ponto de retorno  
  
    move    $ra, $s0  
    li      $v0, 10  
    syscall
```

CHAMADA DE SUBROTINAS

Exemplo 1 (Passando argumentos)

```
main() {  
    int index;  
    scanf("%d", &index);  
    ImprimeValor(index);  
}  
  
ImprimeValor(int _parametro){  
    printf("Imprimindo valor %d", _parametro);  
}
```



```
...  
ImprimeValor:  
    move    $t0, $a0        # salva valor a ser impresso  
    li      $v0, 4  
    la      $a0, txtImpressao  
    syscall                                # imprime texto  
    li      $v0, 1  
    move    $a0, $t0        # restaura valor a ser impresso  
    syscall  
    jr      $ra
```

CHAMADA DE SUBROTINAS

Exemplo 2 (Retorno de subrotinas)

```
main() {  
    int operadorA;  
    scanf("%d", &operadorA);  
    operadorA = incrementa(operadorA);  
}  
  
incrementa(int _opA){  
    return _opA + 1;  
}
```



```
.data  
    operadorA: .space 4  
.text  
    .globl main  
main:  
    move    $s0, $ra # salva o endereço de retorno  
    li      $v0, 5  
    syscall  
    move    $a0, $v0 # define parâmetro  
    jal     inc      # chama subrotina e salva o ponto de retorno  
    sw      $v0, operadorA($zero)  
    move    $ra, $s0 # restaura ponto de retorno anterior  
    li      $v0, 10  
    syscall  
inc:  
    addi    $v0,$a0,1# retorno feito com o registrador $v0  
    jr      $ra
```

CHAMADA DE SUBROTINAS

Exemplo 2 (Retorno de subrotinas)

```
main() {  
    int operadorA;  
    scanf("%d", &operadorA);  
    operadorA = incrementa(operadorA);  
}  
  
incrementa(int _opA){  
    return _opA + 1;  
}
```



```
.data  
    operadorA: .space 4  
.text  
    .globl main  
main:  
    move    $s0, $ra # salva o endereço de retorno  
    li      $v0, 5  
    syscall  
    move    $a0, $v0 # define parametro  
    jal     inc      # chama subrotina e salva o ponto de retorno  
    sw      $v0, operadorA($zero)  
    move    $ra, $s0 # restaura ponto de retorno anterior  
    li      $v0, 10  
    syscall  
inc:  
    addi    $v0, $a0, 1 # retorno feito com o registrador $v0  
    jr      $ra
```

TRABALHO PRÁTICO

PROJETO

- Escreva um programa em linguagem de montagem do MIPS que leia uma matriz bidimensional de anos (**BISSEXTO_CAND**), e informe quais destes são anos bissextos. A matriz de anos é pré-existente. O programa recebe como entrada o número da linha da matriz que deverá ser alvo da consulta. O programa deve verificar se a linha existe, senão solicita-se um novo número de linha até que esta seja válida. Ao final da análise da linha, a quantidade de anos bissextos encontrados deve ser gravada no espaço de memória **CNT**, e cada ano bissexto encontrado é gravado no vetor **RESULTADO**. Lembre-se que um ano é bissexto se **(ano mod 4 == 0 e (ano mod 100 != 0 ou ano mod 400 == 0))**. Por fim, durante a execução do programa, mantenha o usuário informado do que está sendo computado, via mensagens para este

PROJETO

- Um exemplo de área de dados que se pode utilizar para o programa é

```
.data
BISSEXTO_CAND:
                .word 2016 2012 2008 2004 2000
                .word 1916 1911 1908 1904 1900
                .word 1861 1857 1852 1846 1844
                .word 1728 1727 1726 1725 1723

LINHAS: .word 4
COLUNAS: .word 5
CNT:      .word 0
RESULTADO: .word 0 0 0 0 0
TEXTO_1: .asciiz "Qual linha deseja verificar?"
TEXTO_2: .asciiz "Total de anos bissextos: "
TEXTO_3: .asciiz "\nOs anos bissextos são: "
TEXTO_4: .asciiz ", "
```

PROJETO

- Ao final da execução do programa, o seguinte resultado é esperado para as variáveis destacadas. Neste exemplo, o usuário indicou a linha 0!

```
CNT:          .word 5
```

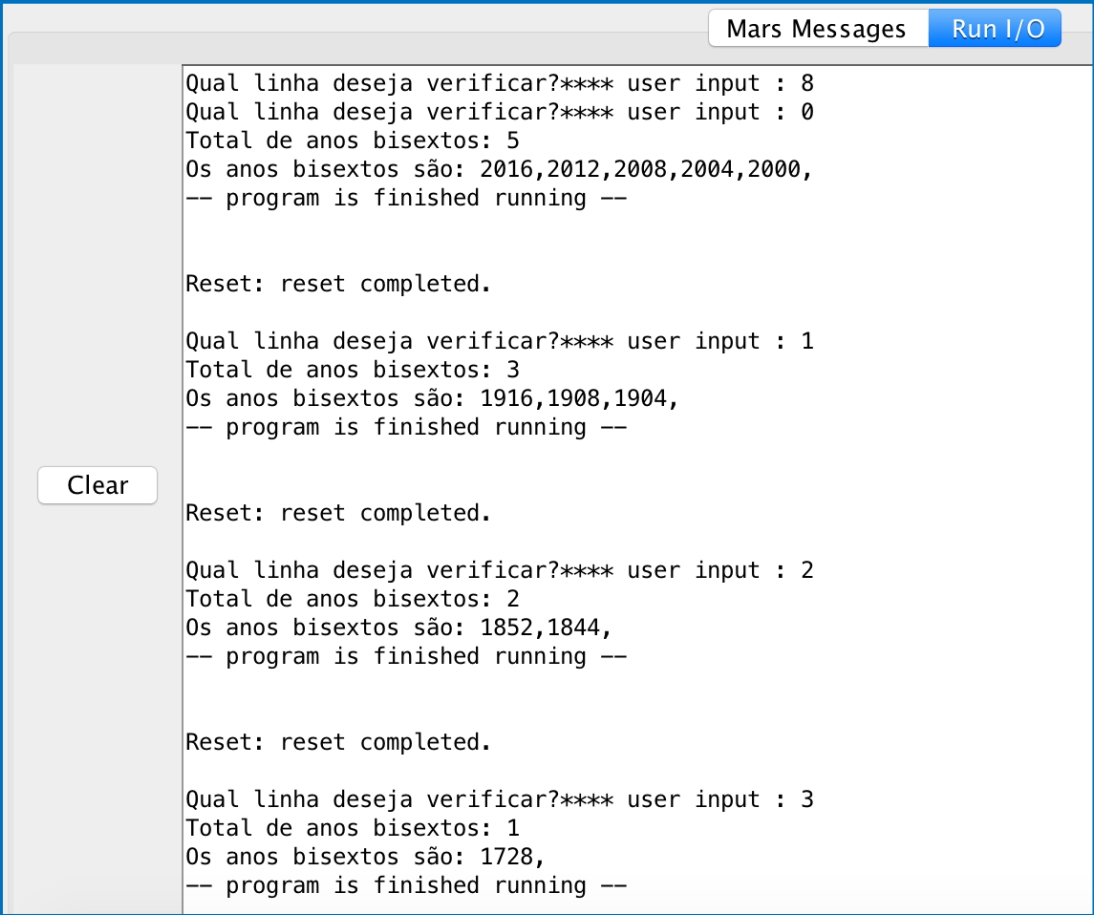
```
RESULTADO:    .word 2016 2012 2008 2004 2000
```

PROJETO – Regras do Jogo

- O programa escrito deve ser independente do tamanho da tabela de anos
 - Ele deve funcionar para qualquer tamanho, desde que se respeite as restrições seguintes
 - LINHAS → numeral Natural >0
 - COLUNAS → numeral Natural >0
- Utilizem sub-rotinas para encapsular as partes do código
 - Partes naturais do programa candidatas a serem implementadas como subrotina são
 - O cálculo da bissexatualidade de um ano, dado o ano
 - A impressão do resultado da computação, dados CNT e o vetor RESULTADO

PROJETO

- A tela ao lado mostra alguns exemplos de execução do programa, realizado no Mars
- Observe-se que a primeira linha indicada é 8, que é inválida por ser maior que o número de linhas da matriz de anos
- Assim, foi pedido que o usuário indicasse outra linha para verificação
- Nas demais simulações, informa-se uma linha válida, e se pode observar a resposta esperada!



The screenshot shows the Mars Messages window with a 'Run I/O' button. The output text is as follows:

```
Mars Messages Run I/O
Qual linha deseja verificar?**** user input : 8
Qual linha deseja verificar?**** user input : 0
Total de anos bisextos: 5
Os anos bisextos são: 2016,2012,2008,2004,2000,
-- program is finished running --

Reset: reset completed.

Qual linha deseja verificar?**** user input : 1
Total de anos bisextos: 3
Os anos bisextos são: 1916,1908,1904,
-- program is finished running --

Clear

Reset: reset completed.

Qual linha deseja verificar?**** user input : 2
Total de anos bisextos: 2
Os anos bisextos são: 1852,1844,
-- program is finished running --

Reset: reset completed.

Qual linha deseja verificar?**** user input : 3
Total de anos bisextos: 1
Os anos bisextos são: 1728,
-- program is finished running --
```